

Modelling Workflows, Service Dependencies and a Group Cloud Case

Adnan Khan

I worked through individual process and architecture models, then contributed to a group cloud-service case. This walkthrough covers the full range of scenarios and all five group model areas: service functions, user goals, architecture, login and subscription.

Tools and environment

Archi; business/application/technology views; UML use cases; process and collaboration models; workflow review.

Individual labs and assignments: Adnan Khan. Group case: Sajan Mahilrajan, Kieran Singh, Kapienath Muthiah and Adnan Khan. Individual ownership of the group diagrams was not recorded. September-November 2025.

Model the home-monitoring response

I modelled a system moving from disarmed to armed operation and responding to a break-in. The notification work covers voice, SMS and fax, followed by the owner response or escalation to emergency services. This describes a proposed workflow rather than a built alarm system.

- Detection leads to the configured notification channels.
- Notification uploads precede waiting for the owner.
- Owner response and no-response escalation are separate outcomes; a final specification needs an agreed timeout and channel-completion rule.

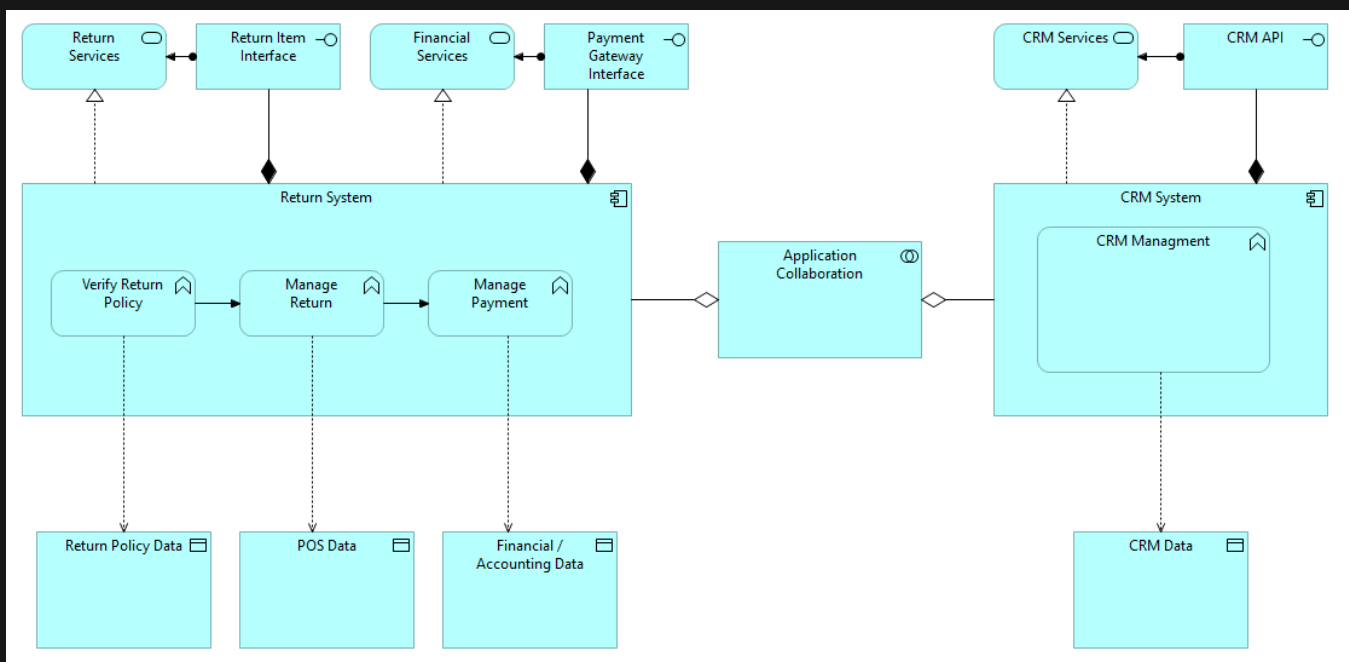
Separate order storage from fulfilment decisions

My order model stores the incoming order while evaluating capacity and parts. Available resources lead toward fulfilment; unavailable capacity leads to rejection; missing parts require procurement. I kept the different business outcomes explicit in this summary.

- The original exercise also includes a four-hour timer/reset branch. A specification must define what happens to the retained order and customer notification on timeout.

Connect returns to supporting applications

I modelled receipt checking, inspection, approval and refund/rejection at the business level. The application view then connects policy verification, returns, payment and CRM responsibilities with their interfaces and data. Together, the views explain the work and the systems behind it.



Return application model. Source portfolio p. 13.

Define banking participants and goals

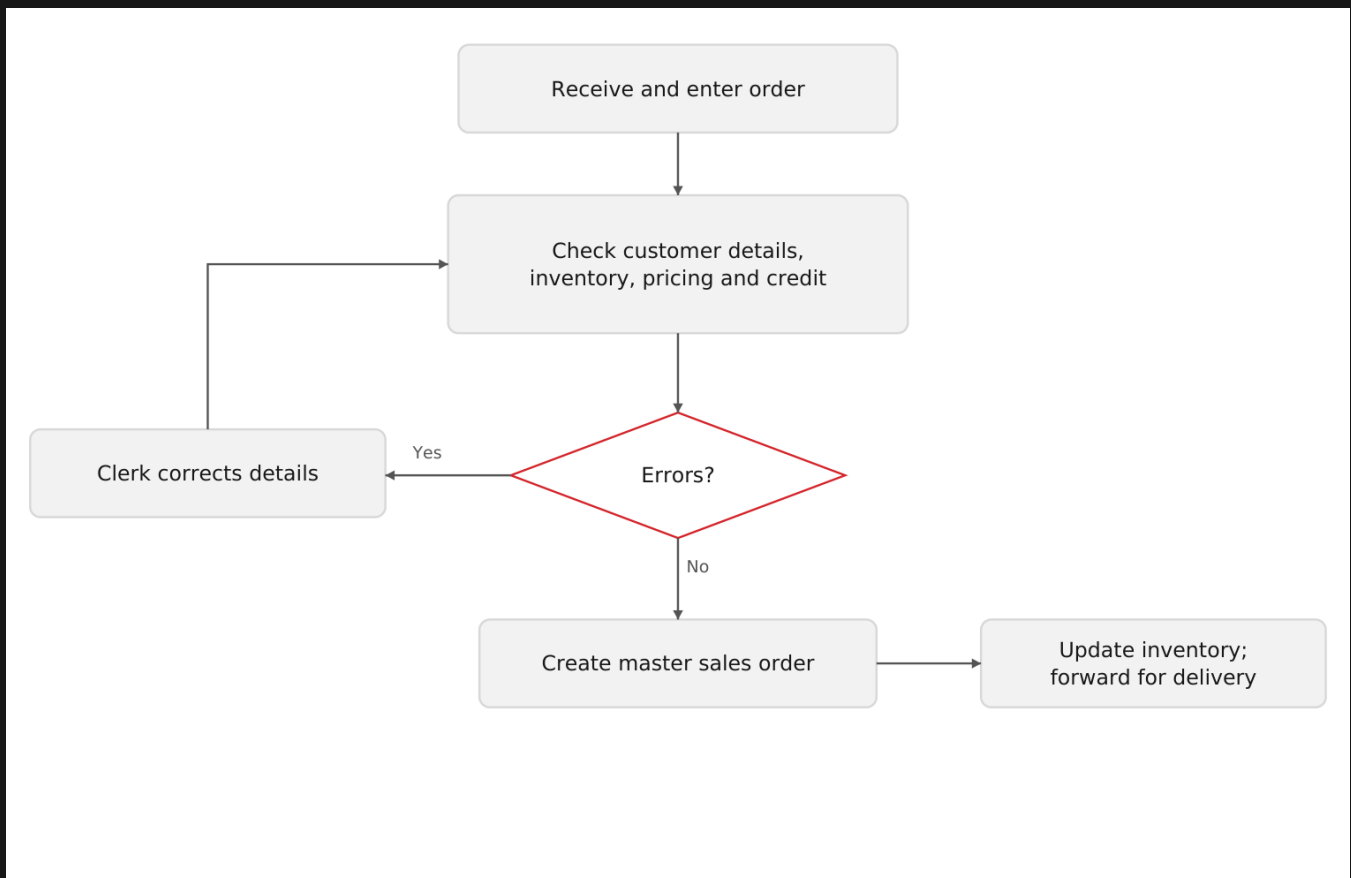
I used a banking-system boundary to identify customer, ATM, cashier and manager interactions. The scope covers accounts, deposits, withdrawals, transfers and loan-related decisions. This is a use-case view of responsibilities, not the execution sequence of a live banking transaction.

- The useful scope question is which participant needs which goal.
- Account verification, balance checks and decision handling should be defined consistently before implementation.

Follow sales orders through validation

My collaboration model follows the customer, sales representative, headquarters, clerk, verification and delivery handoffs. Customer details, inventory, pricing and credit are checked before creating the master sales order and forwarding it for delivery.

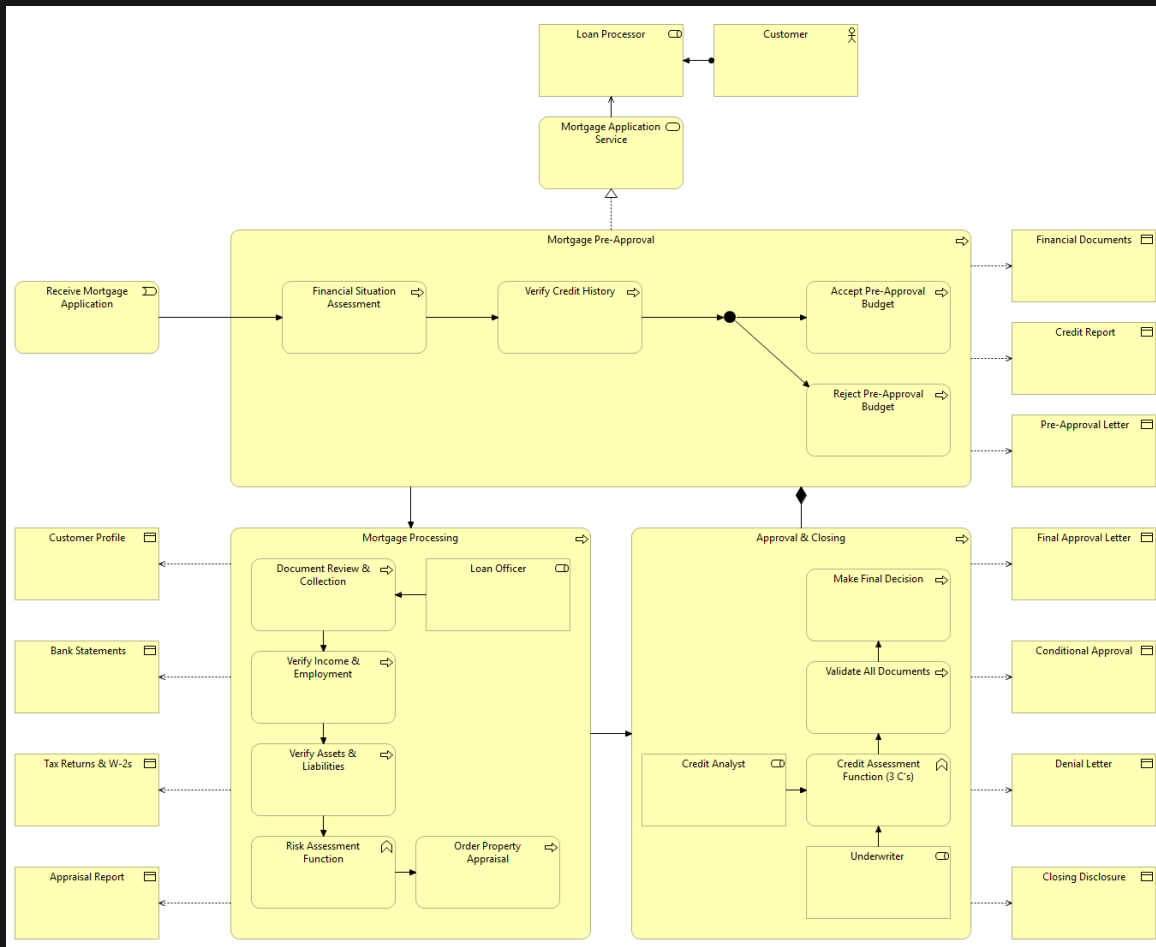
- A corrected review sequence returns errors to the clerk and repeats validation.
- Inventory is updated only after the order passes the required checks.
- The end-of-day handoff belongs to the case workflow; it is not a measured service level.



Revised design summary based on the source portfolio; not an execution screenshot.

Organise mortgage work and supporting records

I grouped the mortgage service into pre-approval, processing, and approval/closing. The activities cover financial checks, employment/income, assets/liabilities, risk and appraisal. Records such as credit reports, bank statements and approval/denial documents explain what each stage needs.

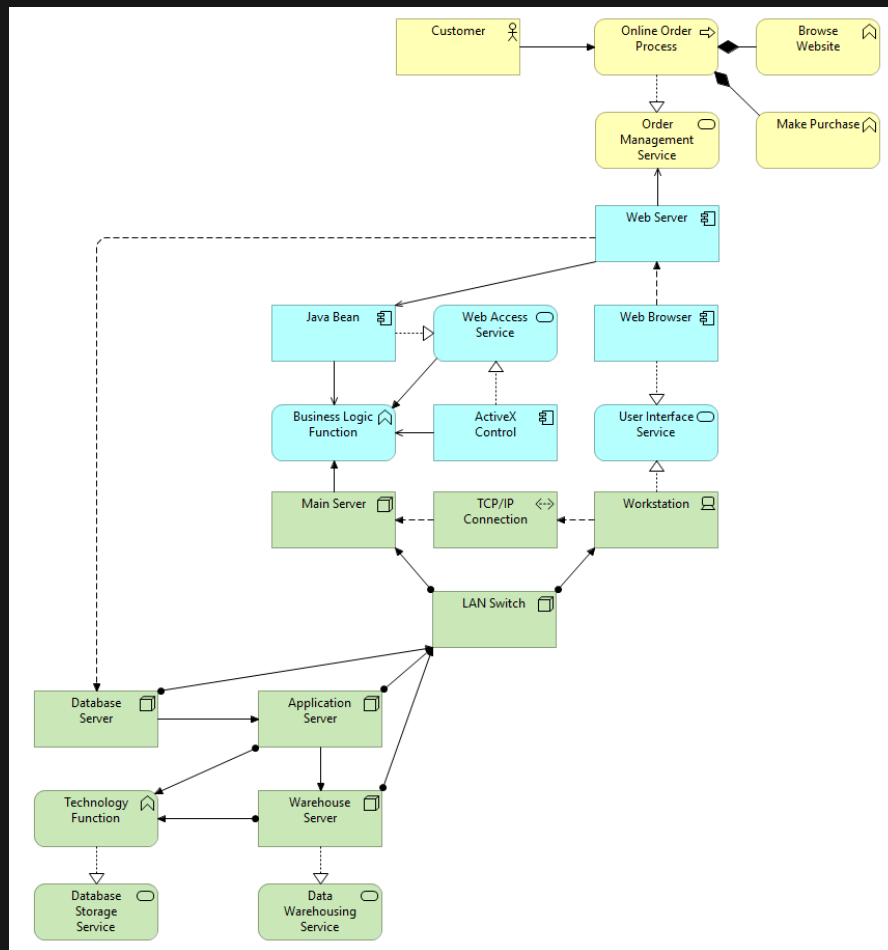


Mortgage business model. Source portfolio p. 22.

Connect online ordering across architecture layers

I used Archi to connect browsing and purchasing to application functions and infrastructure. The submitted view includes web access, business logic, a database server, application server, warehouse server, a LAN switch and TCP/IP connections.

- The technologies belong to the academic model. The value is tracing dependencies and ownership when discussing an unavailable service.

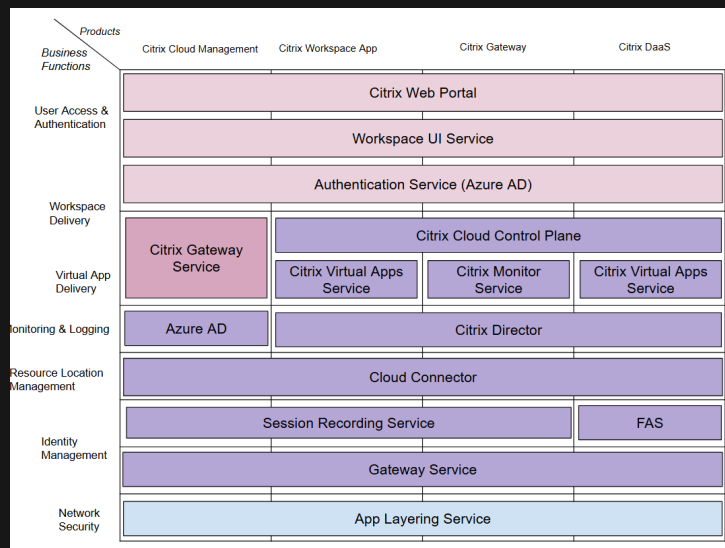


Online order architecture. Source portfolio p. 26.

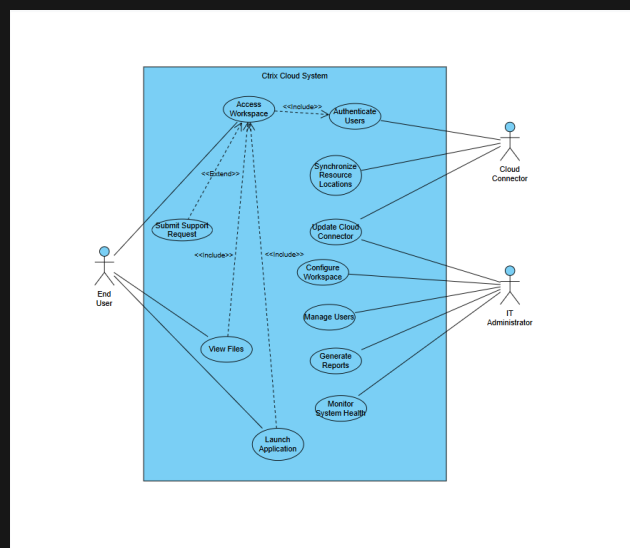
Explain the group cloud-service scope

Our group examined a Citrix Cloud case through service/function mapping and user goals. The service view organizes the business functions against the cloud components named in the submission. The use cases identify end-user, administrator and Cloud Connector interactions.

- The scope includes application access, user management, reporting and system-health monitoring.
- The service mapping is an academic model, not a vendor-validated configuration or a deployed Citrix environment.



Team evidence: Cloud service mapping. Source portfolio p. 28.

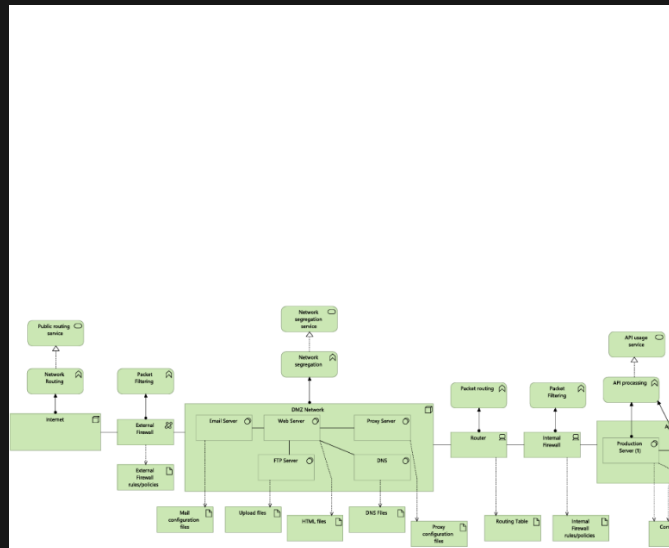


Team evidence: Cloud system use cases. Source portfolio p. 29.

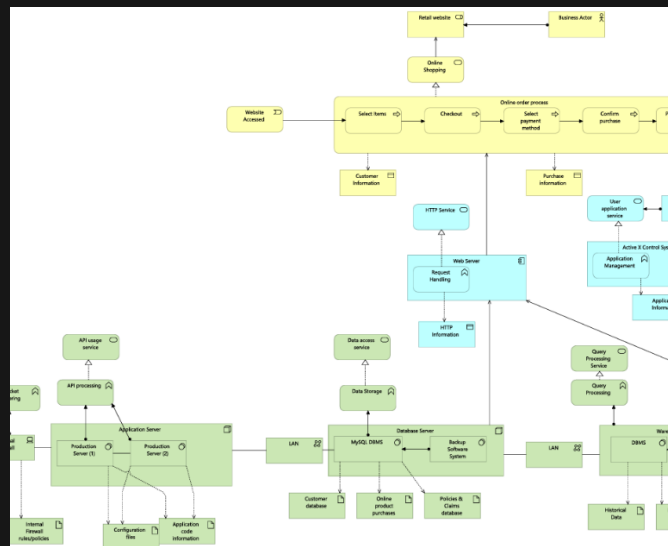
Review the group's architecture study

The architecture supplied in our group report models a retail website and online ordering with Internet access, firewalls, DMZ services and internal servers. I present it as the group's retail dependency study. It helps explain network boundaries and application/server dependencies.

- The drawing does not establish a configured connection between that retail stack and Citrix services. I do not label it as a Citrix deployment architecture.



Team evidence: Group architecture overview detail. Source portfolio p. 32.

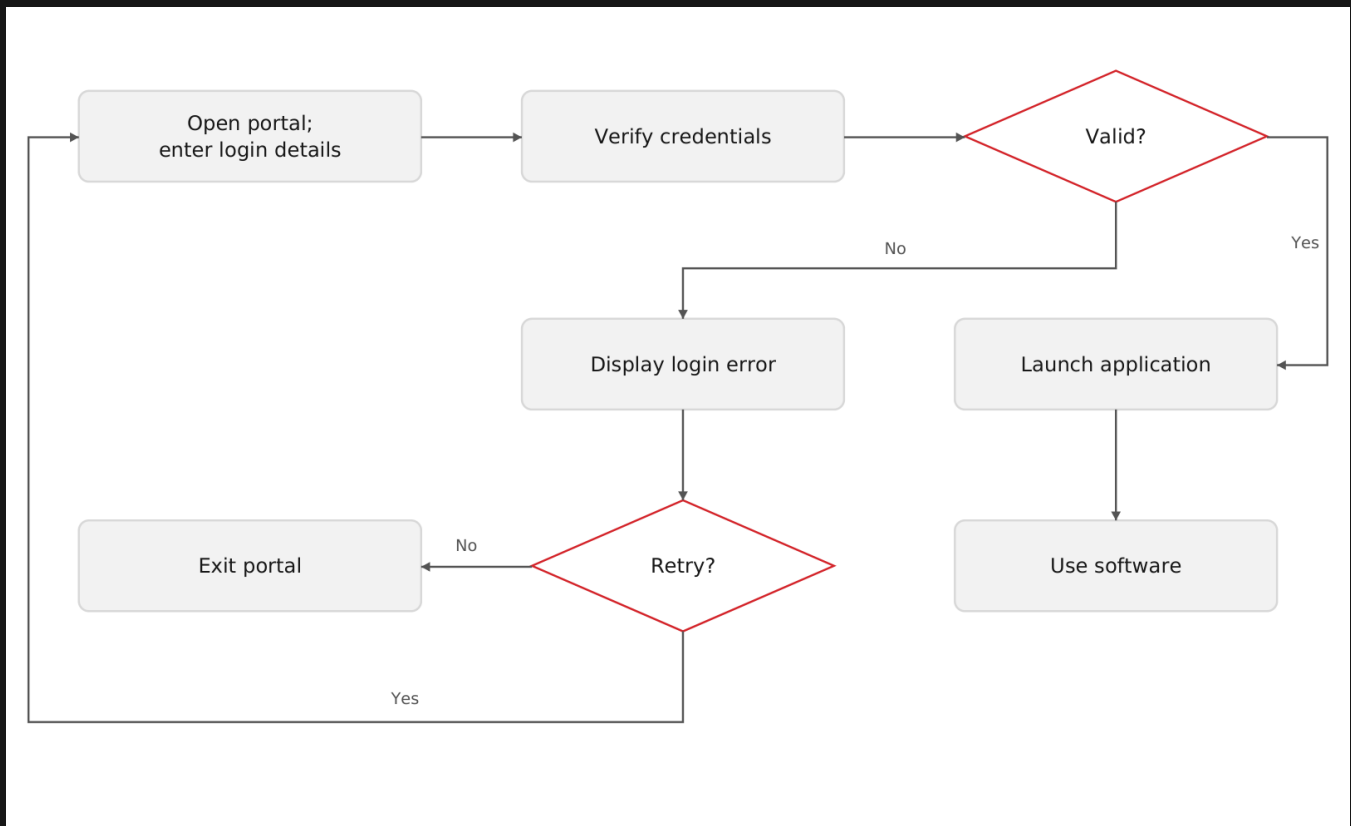


Team evidence: Group architecture overview detail. Source portfolio p. 33.

Model cloud login and retry outcomes

Our group access flow starts at the portal, checks credentials and leads accepted users to application launch. The clarified sequence sends invalid entries to an error and then a retry or exit. This makes the unsuccessful path as visible as the successful one.

- Portal → enter login details → verify credentials.
- Accepted → launch application → use software.
- Rejected → display error → retry input or exit.

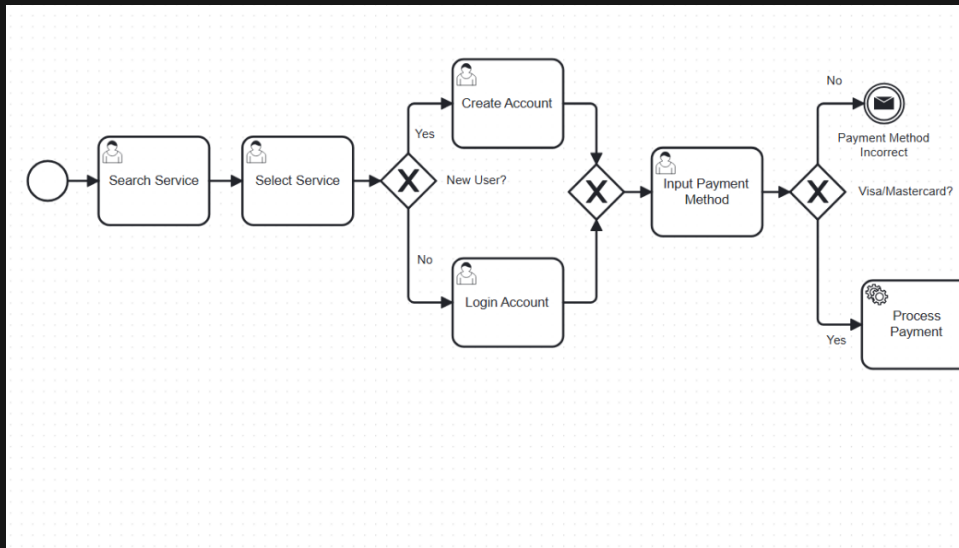


Revised design summary based on the source portfolio; not an execution screenshot.

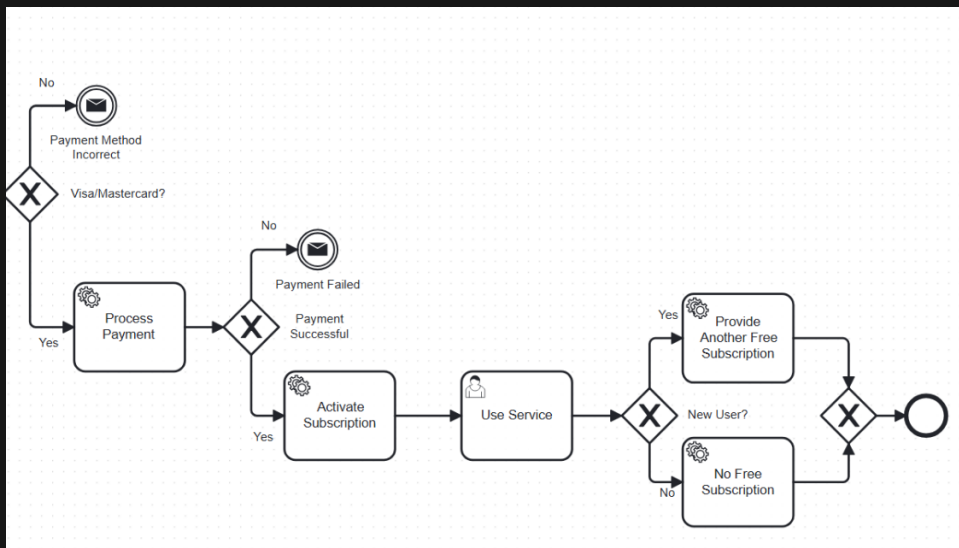
Follow the entire subscription journey

Our subscription model begins with finding and selecting a service. New users create an account; existing users sign in. The flow checks payment details, processes payment and activates the subscription on success, with invalid-payment and payment-failure outcomes.

- The later new-user offer branch is a case assumption. The model is a design artifact, not evidence of payment settlement.



Team evidence: Subscription workflow detail. Source portfolio p. 38.



Team evidence: Subscription workflow detail. Source portfolio p. 39.

Review normal paths, exceptions and dependencies

I reviewed whether each scenario identifies participants, decisions, required records and outcomes. Normal and unsuccessful paths need the same attention. Across views, names and responsibilities should remain consistent so a process step can be traced to its supporting service.

- Scenario reviews include unavailable parts, rejected returns, invalid credentials and failed payment.
- A model walkthrough is different from running an implementation test.
- The next implementation stage would need explicit recovery rules, access controls and acceptance evidence.

What I learned

- A business view explains work; an application view explains software responsibilities.
- A useful workflow includes both successful and unsuccessful outcomes.
- Label a model by what it actually contains and keep proposed architecture separate from deployment evidence.

Source: the full ITM706 portfolio supplied for this project. Evidence is from the original coursework; this edition adds no new lab test results. Repeated captures and earlier drafts are consolidated. Revised design summaries are labelled, and sensitive identifiers are masked.